

Mohamed Shafi Uzman Fassy – (102608927)

COS30031 – Task 6 – 4th Oct 2023

Q.1 [line 104] There are inside array_demo_1 - answer them there.

Q.1.1 [line 166] What do the < and > mean or indicate?

1.1. They are used to specify template arguments in C++. With `std::array`, they define the type and fixed size of the array.

Q.1.2 [line 167] Why don't we need to write `std::array` here? (Is this good?)

1.2. Using namespace `std`; declaration allows using `std` objects without the prefix. While it simplifies code, it can introduce ambiguity.

Q.1.3 [line 168] Explain what the `int` and `3` indicate in this case?

1.3. In the declaration `array<int, 3> a1`, `int` is the data type of the elements that `a1` will hold, and `3` is the fixed size of the array.

Q.1.4 [line 206] In the code above, what is the type of `itr2`?

1.4. The type of `itr2` is an iterator to the `std::array<int, 3>`. Specifically, it's `std::array<int, 3>::iterator`. The use of `auto` lets the compiler deduce this type automatically.

Q.1.5 [line 213] In the code above, what is the type of `v`?

1.5. "`v`" is of type `int`, which is the element type of the array `a1`.

Q.1.6 [line 214] In the code above, what does the `&` mean in `(auto &v : a1)`

1.6. The `&` means that `v` is a reference to the elements in `a1`. This allows for direct modification of the elements in the array without making a copy.

Q.1.7 [line 222] Try this. Why does `a1[3]` work but `at(3)` does not?

1.7. Because, `a1[3]` accesses the array without checking boundaries, which can be risky. Meanwhile, `a1.at(3)` checks if the index is valid and throws an error if it's out of bounds. The `at()` method is safer because of this check.

Q.1.8 [line 235] `auto` is awesome. What is the actual type of `v` that it works out for us?

1.8. The for loop iterating over `a1`, the type of `v` is `int&` and basically references to `int`.

Q.1.9 [line 242] `auto` is still awesome. What is the actual type of `v` here?

1.9. the for-each loop iterating over `a1`, the type of `v` is `int&`.

Q.1.10 [line 252] How would you do a forward (not reverse) sort?

1.10. You could use the `sort` function with the `begin` and `end` iterators of the array: `sort(a1.begin(), a1.end());`.

Q.2 [line 107] In `array_demo_2`, explain what `a4(a1)` does

Answer: `a4(a1)` initializes the `a4` array by copying the contents of `a1`. It's a copy constructor call.

Q.3 [line 110] No questions for `array_demo_3`, it's just a demo of Struct/Class use with array.

I just ran the a breakpoint on `array_demo_3` just to see what it does.

Q.4 [line 113] How do we (what methods) add and remove items to a stack?

We use `push()` to add and `pop()` to remove items from a stack.

Q.5 [line 114] A stack has no `[]` or `at()` method - why?

A stack is designed as a Last-In-First-Out (LIFO) data structure, meaning you only have access to the top element. Direct indexing would violate this principle.

Q.6 [line 117] What is the difference between a `stack.pop()` and a `queue.pop()` ?

A `stack.pop()` removes the item from the top (last added item), whereas a `queue.pop()` removes the item from the front.

Q.7 [line 120] Can we access a list value using an int index? Explain.

Not really because lists in C++ are doubly-linked lists and they do not support direct indexing like arrays because accessing elements requires traversal from the beginning or end of the list.

Q.8 [line 121] Is there a reason to use a list instead of a vector?

There is, because lists provide efficient insertions and deletions in the middle, whereas vectors are efficient for random access. Use a list if you need frequent insertions/deletions not at the end.

Q.9 [line 124] Was `max_size` and `size` the same? (Can they be different?)

No, they were different as `size` returns the current number of elements, while `max_size` returns the maximum number of elements a container can accommodate due to the system or the library implementation limitations.

Q.10 [line 125] Which `ParticleClass` constructor was called?

Both the constructors were called as the default constructor was called when the `std::array<ParticleClass, 3> a1;` is defined, and then the parameterized constructor was called when adding items using `push_back`.

Q.11 [line 126] Were the `ParticleClass` instances deleted? If so, how?

Yes, they were when they went out of scope the container holding them was destroyed as the destructor was invoked for each instance.

Q.12 [line 127] Was the vector instance deleted? If so, how do you know this?

Yes, when the function or scope in which the vector was defined ends, the vector's destructor was called and which in turn calls the destructors of its elements.

Q.13 [line 128] Your IDE might suggest to use `emplace_back` instead of `push_back`. What does this mean?

My IDE (Visual Code Studio 2023) did not suggest to use “`emplace_back`” constructs over “`push_back`” but from what I researched `emplace` is new in C++ 11 and It seems to be often more efficient when adding objects to a container since it can directly take constructor arguments and avoid temporary object creation.